Version 3 : Optimized GPU Implementation

Overview

The optimized GPU implementation of focuses on improving performance through kernel tuning, memory hierarchy usage, and efficient data communication. The key optimization areas are explained below.

(a) Launch Configuration Optimization

- **Block Size:**

  The kernel uses a block size of **16 × 16 threads (256 threads per block)**.

  This is an optimal size that balances computation and resource usage across GPU cores.

- **Multiple CUDA Streams:**

  4 CUDA streams are used for batched kernel launches.

  Streams allow simultaneous execution of computation and data transfer, increasing GPU utilization.

(b) Communication Optimization

- **Asynchronous Data Transfers:**
  - Data transfers are handled with `cudaMemcpyAsync()` using multiple streams.
  - This allows **data transfer and kernel execution to happen in parallel**.

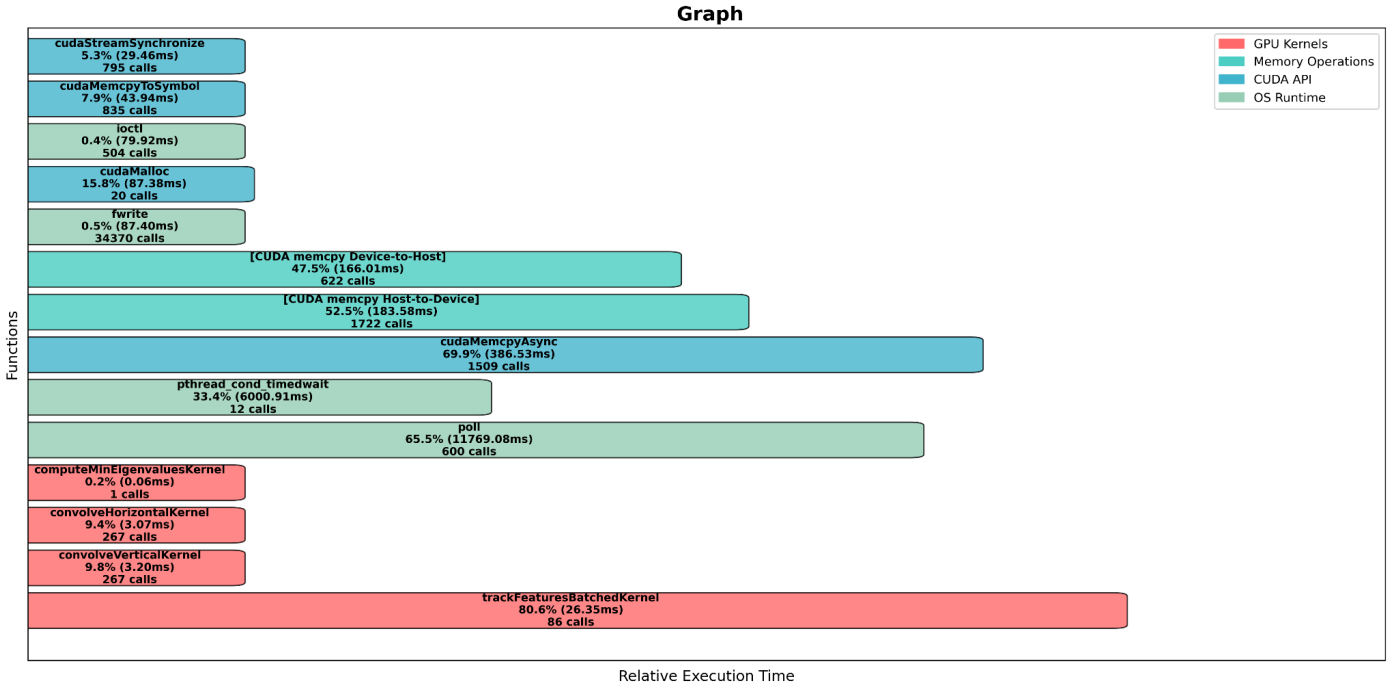
- **Batched Feature Data Structure:**
  - Many small memory copies are grouped into a single large batch transfer.

(d) Memory Optimization

- **Constant Memory Usage:**
  - Parameters like window size, step factor, and thresholds are stored in **`__constant__` memory**.
  - All threads can quickly read these values from the cached constant memory.

- **Memory Pool / Buffer Reuse:**
  - Image buffers (`d_img_buffer`, `d_diff_buffer`, etc.) are reused instead of reallocated.

Profiling Results



Main GPU Kernel (`trackFeaturesBatchedKernel`)

- Accounts for **~80%** of total GPU execution time — meaning computation is now fully GPU-bound rather than transfer-bound.

Memory Transfers (`cudaMemcpyAsync`, Host↔Device)

- Reduced relative cost due to asynchronous overlapping with computation.

Reduced CUDA API Overhead:

- Fewer calls to `cudaMalloc` and `cudaFree` indicate effective memory reuse.